

A-FedHealth: Privacy-Preserving Medical Diagnosis Using Federated Learning

1. Introduction

This project is about building a smart medical AI system that can learn from many hospitals without sharing patient data.

Normally, to train AI:

- All hospitals send patient data to one place.
- AI is trained there.
- This is risky and unsafe.

In this project:

- Each hospital keeps its own data.
- AI learns inside each hospital.
- Only learning results are shared.

So, patient data is always safe.

2. Problem Statement

Today, hospitals face these problems:

- Patient data is private.
- Laws do not allow hospitals to share data.
- Small hospitals have less data.
- AI trained in one hospital may not work well in others.

Because of this:

- AI becomes biased.
- Diagnosis may be wrong.
- Patients can be affected.

There is no safe way for hospitals to train AI together.

3. Proposed Solution

This project uses Federated Learning.

Federated Learning means:

“Train AI in many places, but never move the data.”

Instead of sending data to AI:

- AI goes to the data.
- Learns locally.
- Sends only learning results.

Main idea:

- Data stays in hospitals.
 - Knowledge is shared.
 - Privacy is protected.
-

4. Objectives of the Project

The main goals are:

- To protect patient privacy.
 - To improve medical AI accuracy.
 - To avoid data sharing.
 - To reduce bias in AI models.
 - To follow privacy laws.
 - To build a distributed system.
-

5. System Features

Phase 1: Local Hospital System

- Each hospital has its own server.
 - Stores patient records.
 - Stores medical images.
 - Manages local database.
-

Phase 2: Local AI Training

- AI model is trained inside each hospital.
- Uses local X-ray or MRI images.

- No data leaves the hospital.
-

Phase 3: Secure Model Sharing

- Hospitals send only model weights.
 - No patient data is sent.
 - Data is encrypted for safety.
-

Phase 4: Global Model Update

- Central server collects weights.
 - Combines them.
 - Creates better AI model.
 - Sends it back to hospitals.
-

6. System Architecture

High-Level Flow:

Hospital Web Server (Spring Boot)

↓

Local Python AI Trainer

↓

Encrypted Weights

↓

Central Aggregator Server

↓

Updated Model

↓

Hospitals

Main Components

- 1. Hospital Node (Spring Boot)**
 - Manages patients.
 - Handles APIs.
 - Controls training requests.
- 2. AI Engine (Python)**

- Trains neural networks.
- Processes medical images.
- Generates weights.

3. Central Server

- Collects updates.
- Averages models.
- Distributes new model.

4. Database

- Stores patient metadata.
- Stores training logs.

5. Security Layer

- Encrypts communication.
 - Protects model updates.
-

7. Working Methodology

Step 1: Hospital Setup

- Create multiple hospital servers.
- Each has its own dataset.

Step 2: Local Training

- Server starts AI training.
- Python trains model locally.

Step 3: Send Learning Results

- Only weights are sent.
- Data remains private.

Step 4: Model Aggregation

- Central server combines weights.
- Calculates average.

Step 5: Send Back Model

- Updated model sent to hospitals.

Step 6: Repeat Process

- Training continues.
 - Model improves each round.
-

8. Research Phase (Semester 1)

In this phase, you study and analyze.

Main Activities:

- Learn federated learning.
- Study privacy methods.
- Compare centralized and federated systems.
- Analyze communication cost.
- Test basic models.

Research Topics:

- Federated Learning in Healthcare.
- Privacy-Preserving AI.
- Distributed Machine Learning.
- Medical Image Analysis.

Output:

- Literature review.
 - Comparison report.
 - Research paper.
 - Basic prototype.
-

9. Execution Phase (Semester 2 and 3)

In this phase, you build the full system.

Main Activities:

- Build hospital servers.
- Connect Python training.
- Develop central server.

- Implement encryption.
- Add monitoring.
- Test performance.

Deliverables:

- Working federated system.
 - Multi-node demo.
 - Secure communication.
 - Final project report.
-

10. Technology Stack

Backend (Hospital Server)

- Java Spring Boot
- REST APIs
- WebSockets / gRPC

AI Engine

- Python
- PyTorch / TensorFlow
- TensorFlow Federated (Optional)

Database

- MySQL / PostgreSQL

Communication

- WebSockets / gRPC
- HTTP APIs

Security

- AES Encryption
 - SSL/TLS
-

11. Advantages of the System

- Protects patient privacy.

- Follows medical laws.
 - Improves AI accuracy.
 - Reduces data misuse.
 - Works in many hospitals.
 - Scalable system.
 - Real-world usable.
-

12. Applications

- Government hospitals.
 - Private hospitals.
 - Medical research centers.
 - Diagnostic labs.
 - Telemedicine platforms.
-

13. Future Scope

- Add more hospitals.
 - Support more diseases.
 - Real-time diagnosis.
 - Mobile doctor app.
 - Cloud deployment.
 - Blockchain audit trail.
-

14. Expected Outcomes

- Secure medical AI system.
- Better disease detection.
- Privacy-safe learning.
- Research publication.
- Industry-level prototype.
- Strong resume project.

